

Sealed Classes:

- Sealed class is declared using sealed keyword.
- Sealed classes allow to declare which class can be a subtype using permits keyword.
- A class extending sealed class must be declared as either sealed, non-sealed or final.
- Sealed classes helps in creating a finite and determinable hierarchy of classes in inheritance.

```
//program for sealed classes
sealed class A permits B, D {
    //body of Class A
}
sealed class B extends A permits C {
    //body of Class B
}
non - sealed class C extends B {
    //body of Class C
}
non - sealed class D extends A {
    //body of Class C
}
class SealedDemo {
    public static void main(String args[]) {
        //body of Main Function
    }
}
```

Example:

```
//program for Multilevel using sealed
sealed class A permits B {
    int a;
    A(int x) {
        a = x;
    }
    void display() {
        System.out.println("ClassA A=" + a);
    }
}
sealed class B extends A permits C {
    int b;
    B(int x, int y) {
        super(x);
        b = y;
    }
    void show() {
        System.out.println("ClassB =" + (a + b));
    }
}
```

```

    }
}
non - sealed class C extends B {
    int c;
    C(int x, int y, int z) {
        super(x, y);
        c = z;
    }
    void view() {
        System.out.println("ClassC =" + (a + b + c));
    }
}
}
class SealedClass {
    public static void main(String args[]) {
        C obj = new C(12, 16, 28);
        obj.display();
        obj.show();
        obj.view();
    }
}

```

Output:

```

ClassA A=12
ClassB =28
ClassC =56

```

Default methods in interfaces :

Before Java 8, interfaces could have only abstract methods. The implementation of these methods has to be provided in a separate class. So, if a new method is to be added in an interface, then its implementation code has to be provided in the class implementing the same interface. To overcome this issue, Java 8 has introduced the concept of default methods which allow the interfaces to have methods with implementation without affecting the classes that implement the interface

Example:

```

interface TestInterface {
    // abstract method
    public void square(int a); // default method
    default void show() {
        System.out.println("Default Method Executed");
    }
}
class TestClass implements TestInterface {
    // implementation of square abstract method
    public void square(int a) {
        System.out.println(a * a);
    }
}

```

```

    }
    public static void main(String args[]) {
        TestClass d = new TestClass();
        d.square(4);
        // default method executed d.show();
    }
}

```

Output:

16

Default Method Executed Default methods are also known as defender methods or virtual extension methods.

Static methods in interfaces:

The interfaces can have static methods as well which is similar to static method of classes.

Example:

```

interface TestInterface { // abstract method
    public void square(int a); // static method
    static void show() {
        System.out.println("Static Method Executed");
    }
}
class TestClass implements TestInterface {
    // Implementation of square abstract method
    public void square(int a) {
        System.out.println(a * a);
    }
    public static void main(String args[]) {
        TestClass d = new TestClass();
        d.square(4);
        // Static method executed
        TestInterface.show();
    }
}

```

Output:

16
Static Method Executed